

How Delta Lake Computes the Latest State

Delta Lake computes the latest state of a table by reading the transaction log JSON files sequentially and applying add and remove actions to determine which data files are active.

Step-by-Step Execution Flow

1. Table Creation

- A Delta table is created.
- Transaction log initialized as:

000.json

2. Insert Operation

- 5 records are inserted using a single insert statement.
- A Parquet file is created:

file1.parquet

- Transaction log update:

001.json → add file1.parquet

3. Update Operation

- One record is updated.
- A dv file is created.
- A new Parquet file is generated:

file2.parquet

- Transaction log update:

002.json

- remove file1.parquet
- add file1.parquet with dv
- add file2.parquet

First SELECT * FROM table

Logs Read

Delta Lake reads:

000.json

001.json

002.json

Actions Interpreted

add file1.parquet ✗

remove file1.parquet ✗

add file1.parquet with dv ✓

add file2.parquet ✓

add file1.parquet ✗

- file1.parquet was added earlier (in 001.json)
- But later it was removed (in 002.json)
- So this original version of file1.parquet is NOT used
- Hence ✗ (inactive)

remove file1.parquet ✗

- This is just a log action, not a data file
- Removed files are not read for queries
- Hence ✗

add file1.parquet with dv ✓

- A new version of file1.parquet was added with dv
- This file was not removed yet at this point
- So it is active
- Hence ✓ (used in query)

add file2.parquet ✓

- file2.parquet was added during the update
- It was not removed yet
- So it is active
- Hence ✓

Result

Active files used for query:

- file1.parquet (with dv)
- file2.parquet

4. Overwrite Operation

- A new Parquet file is created:

file3.parquet

- Transaction log update:

003.json

- remove file1.parquet with dv
- remove file2.parquet
- add file3.parquet

Second SELECT * FROM table

Logs Read

Delta Lake reads:

000.json

001.json

002.json

003.json

Final Action Resolution

add file1.parquet	X
remove file1.parquet	X
add file1.parquet with dv	X
add file2.parquet	X
remove file1.parquet with dv	X
remove file2.parquet	X
add file3.parquet	✓

Result

- Only one active file:
 - file3.parquet

How Delta Lake Computes the Latest State

Based on the operations shown:

1. Delta Lake reads all JSON transaction logs in order.
2. Each log contains add and remove actions.

3. Files that are removed later are ignored.
4. Queries use only the files that remain active at the end.

Logical Flow Diagram

000.json
↓
001.json → add file1.parquet
↓
002.json → remove file1.parquet
 add file1.parquet with dv
 add file2.parquet
↓
003.json → remove file1.parquet with dv
 remove file2.parquet
 add file3.parquet

Time Travel and Retention of Parquet Files

Delta Lake retains Parquet files that are no longer part of the latest table state to support **time travel**, which allows querying the table as it existed at a previous version or point in time.

When files are marked as **removed** in the transaction logs, they are excluded from the latest queries but are not immediately deleted from storage. The transaction log records both the **addition time** and **removal time** of each file. Using this information, Delta Lake can reconstruct the table state for any historical version or timestamp.

Version and Time Representation

The relationship between table versions and time can be represented as:

V1 → T1
T1.5
V2 → T2
V3 → T3

- **V1, V2, V3** are committed table versions.
- **T1, T2, T3** are the timestamps at which those versions were created.
- **T1.5** represents a point in time that lies *between* version V1 and V2.

If a query refers to a time such as **T1.5**, Delta Lake resolves it by referring to the **most recent version that was committed before that time**, which is **V1**.

This means that any time-based reference between V1 and V2 will return the table state as of **V1**, since no new version was committed during that interval.

Table State Reconstruction

- At **V1 (T1)**, the table reflects the state after the first set of operations.
- At **T1.5**, the table state remains the same as **V1**, since no new version was committed.
- At **V2 (T2)**, changes committed after V1 result in a new table state.
- At **V3 (T3)**, further changes produce another version.

To answer a time-travel query, Delta Lake scans the transaction logs up to the specified version or time and includes only the Parquet files that were **active at that point**.

Relationship Between Time Travel and File Retention

Parquet files that were added in earlier versions and removed in later versions are retained so that:

- Historical versions of the table can be queried.
- The exact table state can be reconstructed for any version (V1, V2, V3) or time (such as T1.5).

This retention of Parquet files, combined with the transaction log, enables Delta Lake to support time travel while still computing the latest table state accurately.

Vacuum and Data File Cleanup

Delta Lake retains Parquet files to support time travel, but files that are no longer referenced by any valid table version are eventually cleaned up using VACUUM.

VACUUM is a maintenance command that removes data files which are no longer referenced by the Delta table older than a specified retention period. By default, this retention period is 7 days.

Purpose of VACUUM

Running VACUUM regularly is important:

- To save storage cost by deleting unused data files
- For compliance reasons, where old data must not be retained indefinitely

VACUUM only removes files that:

- Are no longer referenced by any Delta table version
- Are older than the configured retention duration

Predictive Optimization (VACUUM)

VACUUM can be part of predictive or automated optimization strategies to keep Delta tables clean and cost-efficient by periodically removing obsolete files.

Retention Duration Configuration

The retention period for deleted files can be configured at the table level:

```
ALTER TABLE tablename SET TBLPROPERTIES (  
'delta.deletedFileRetentionDuration' = '30 days'  
);
```

This configuration controls how long deleted files are retained before VACUUM can remove them.

Retention Safety Check

Delta Lake enforces a safety check to prevent accidental data loss when using very low retention periods. This check can be disabled using:

```
spark.conf.set(  
"spark.databricks.delta.retentionDurationCheck.enabled",  
"false"  
)
```

Disabling this check allows VACUUM to run with aggressive retention settings.

Example: Folder Cleanup Behavior

Consider the following directory structure before running VACUUM:

```
folder1 → files 1, 2, 3  
folder2 → file 4  
folder3 → files 5, 6
```

If file 4 is no longer referenced by the Delta table and exceeds the retention period:

VACUUM (DRY RUN)

- File 4 is identified as eligible for deletion

Directory state remains unchanged during dry run.

VACUUM (Actual Run)

After running VACUUM:

folder1 → files 1, 2, 3

folder2 → (empty)

folder3 → files 5, 6

VACUUM may leave empty directories.

Running VACUUM again can result in:

folder1 → files 1, 2, 3

folder3 → files 5, 6

The empty directory is removed.

Retain 0 Hours

Using retain 0 hours was discussed as a special case.

Example command:

VACUUM tablename RETAIN 0 HOURS; Explanation:

- This command removes all data files that are not referenced by the latest table version
- Older versions become inaccessible immediately

Why RETAIN 0 HOURS Should Not Be Used in Production

- It breaks time travel completely
- Any query referring to older versions or timestamps will fail
- Accidental deletes or updates cannot be recovered
- It can lead to irreversible data loss if run unintentionally

Because of these risks, retaining 0 hours is not recommended in production environments.

VACUUM should always be used with a safe retention period that balances storage cost, compliance needs, and recovery requirements.

Delta Log Files and Read Performance

Delta Lake uses JSON transaction log files to track all changes to a table, including adds, removes, and updates. With multiple JSON files over time, the challenge shifts from storage cost to query read performance.

Log File Growth Example

Consider a table where:

- 100 Parquet data files exist
- 100 JSON transaction log files exist

If only a few records are inserted per operation, many JSON files are created:

- Example: 8 records inserted
 - 8 JSON files are created
 - Some may be combined into a single JSON file (1-6 combined)
 - 7.json, 8.json remain individually

After multiple operations, the table may have versions like:

80th version:

00000000000000000000000072.checkpoint.parquet

00000000000000000000000073.00000000000000000000000078.compacted.json

79.json

80.json

A sample table location for logs:

https://ttmystorageaccount001.blob.core.windows.net/externaldata/orders/ delta_log/00000000000000000000000073.00000000000000000000000078.compacted.json

Querying a Table

Running a query such as:

SELECT * FROM TABLE;

Delta Lake reads checkpoint and compacted JSON files sequentially:

00000000000000000000000072.checkpoint.parquet

00000000000000000000000073.00000000000000000000000078.compacted.json

00000000000000000000000079.00000000000000000000000084.compacted.json

00000000000000000000000085.00000000000000000000000090.compacted.json

00000000000000000000000091.00000000000000000000000096.compacted.json

97.JSON

98.JSON

99.JSON

Log and Parquet File Retention

Delta Lake configuration properties control retention:

- `delta.logRetentionDuration` – controls how long JSON log files are kept
- `delta.deletedFileRetentionDuration` – controls how long deleted Parquet files are kept
`delta.deletedFileRetentionDuration = "interval 30 days"`
- In default settings, `delta.deletedFileRetentionDuration` aligns with `delta.logRetentionDuration`

Key Considerations

- Over time, JSON log file growth can slow down read performance, especially with many small commits
- Checkpoints and compacted log files help reduce read overhead
- Regular maintenance and compaction can optimize query performance while retaining time-travel capabilities

Upsert / MERGE in Delta Lake

Upsert is a DML operation that combines:

- UPDATE (for existing records)
- INSERT (for new records)

In Delta Lake, upsert logic is implemented using the MERGE operation.

Core Concept

- Incoming data arrives as a batch (source)
- Existing Delta table acts as the target
- Each incoming record is checked against the target using a matching key
- Based on the match:
 - Existing records are updated
 - New records are inserted

Incoming Source Data

|

| match on order_id



Target Table (orders)

If match exists → UPDATE

If match not found → INSERT

Upsert ensures that updates and inserts happen in a single operation, making it efficient and consistent when processing incremental or streaming data.

Constraints, ACID & Optimistic Concurrency Control in Delta Lake

Constraints in Delta Tables

Delta Lake supports data constraints to ensure data quality and correctness at write time.

Common Constraints

- **NOT NULL constraint**
 - Example: order_id NOT NULL
 - Ensures every record has a valid order identifier.
- **CHECK constraint**
 - Example: qty > 0, unit_price > 0
 - Ensures only valid business values are written to the table.

If incoming data violates these constraints, the write operation fails.

How Updates and Inserts Work Internally

Delta Lake stores data as Parquet files and tracks changes using transaction logs.

- **INSERT operation**
 - Creates new Parquet files
 - Adds metadata entries for those files
- **UPDATE operation**
 - Does not modify existing Parquet files directly
 - Marks affected rows as deleted (using deletion vectors or file replacement)
 - Writes new Parquet files with updated data

This immutability design enables reliability, time travel, and concurrency control.

ACID Properties in Delta Lake

Delta Lake provides full ACID guarantees on data stored in object storage.

Atomicity

- A transaction is all-or-nothing
- Partial writes are never visible

Consistency

- Table constraints and schema rules are always enforced
- Data moves from one valid state to another valid state

Isolation

- Concurrent operations do not corrupt data
- Readers always see a consistent snapshot of the table

Durability

- Once a transaction is committed, it remains persisted
- Data survives failures and restarts

Isolation and Concurrency Control

Isolation defines how concurrent reads and writes are handled.

Pessimistic Concurrency Control

- Uses row-level or table-level locks
- Writers block other writers
- Common in traditional RDBMS systems
- Not scalable for distributed systems

Optimistic Concurrency Control (Used by Delta Lake)

Delta Lake uses optimistic concurrency control (OCC).

How OCC Works

- Writers do not lock data upfront
- Each writer:
 1. Reads a snapshot of the table at a specific version
 2. Performs changes independently

3. Tries to commit changes

If the table has changed since the snapshot was read, the commit fails and must be retried.

Version-Based Example

- Table is at version 3

Writer 1

- Starts at time T1
- Reads version 3
- Writes data and commits
- Table moves to version 4

Writer 2

- Starts at time T2
- Still reading version 3
- Tries to commit changes
- Delta detects version mismatch
- Transaction fails due to conflict

Writer 2 must re-read the latest version and retry.

Why Optimistic Concurrency Control Works Well

- No locking overhead
- High scalability for distributed workloads
- Safe concurrent writes
- Strong consistency guarantees

Schema Enforcement & Schema Evolution in Delta Lake

Schema Enforcement

Schema enforcement is the process of strictly maintaining a predefined schema for a Delta table.

- Incoming data must match the existing table schema
- Any data that does not conform is rejected
- Ensures:
 - High data quality
 - Strong consistency

- Reliable downstream analytics

Why Schema Enforcement Matters

- Prevents accidental schema drift
- Avoids corrupt or inconsistent datasets
- Ensures all writers follow the same contract

By default, Delta Lake enforces schema during writes.

Schema Evolution

→ Schema evolution is the flexible ability to change a table's schema over time without breaking existing data pipelines.

→ It allows Delta tables to adapt to changing business requirements while preserving historical data.

→ Common schema evolution scenarios include:

→ Adding new columns

→ Modifying data types (safely)

Adding New Columns

When new columns appear in incoming data, Delta Lake can automatically evolve the schema.

Configuration

```
.option("mergeSchema", "true")
```

What Happens Internally

- Delta compares the incoming schema with the existing table schema
- New columns are merged into the table definition
- Existing data remains unchanged
- New columns appear as NULL for old records

This allows pipelines to continue running without manual schema changes.

Data Type Widening

Data type widening allows safe expansion of column data types.

Example:

- INT → LONG
- FLOAT → DOUBLE

This is useful when data grows beyond original limits.

Configuration

```
'delta.enableTypeWidening' = 'true'
```

- Only widening conversions are allowed
- Narrowing changes (e.g., STRING → INT) are not permitted automatically
- Prevents data loss while enabling schema flexibility

Rename or Delete a Column in Delta Lake

Why Column Rename/Delete Is Challenging in Parquet

Parquet files store both data and column names together inside the file.

- Column names are embedded in the Parquet file schema
- Traditionally, renaming or deleting a column would require:
 - Reading all data files
 - Rewriting them with a new schema
 - High compute and time cost for large tables

How Delta Lake Solves This Problem

Delta Lake uses column mapping to support column renames and deletes efficiently.

Azure Databricks supports column mapping for Delta Lake tables, enabling:

- Metadata-only operations
- No physical rewrite of Parquet data files

Instead of changing the Parquet files, Delta Lake updates the transaction log (metadata).

Column Mapping Concept

Column mapping separates:

- Physical column name (stored in Parquet files)
- Logical column name (exposed to users)

Example

- Physical column name in Parquet file: qty
- Logical column name exposed to users: quantity

When a column is renamed:

- The physical column name remains unchanged in the data file
- The logical name is updated in the Delta log metadata

Renaming a Column

When a column is renamed:

- Delta Lake creates a new log (metadata) entry
- The mapping between logical and physical names is updated
- No Parquet files are rewritten

This makes column renames:

- Fast
- Scalable
- Safe for large tables

Deleting a Column

When a column is deleted:

- Delta Lake marks the column as logically deleted in metadata
- The column data may still exist physically in Parquet files
- Queries no longer see or access the column

Again, this is a metadata-only change.

Benefits of Column Mapping

- Avoids expensive data rewrites
- Enables schema evolution at scale
- Supports production-grade tables with large data volumes
- Keeps historical data intact for time travel

By using column mapping, Delta Lake allows columns to be renamed or deleted through metadata changes only, without rewriting Parquet files. This provides high performance, flexibility, and safety when evolving schemas in large-scale data systems.

